Casey Barton

Honolulu, HI · (808) 936-5654 · glacier.equilibrium@gmail.com · github.com/GlacierEQ · casey-barton-glaciereq.vercel.app · linkedin.com/in/caseybartonai

PROFILE

Background-agent orchestration is my daily operating pattern: three years as a founder running many AI agents in parallel across a real codebase — real branches, worktrees, and review flow as the native medium. I own customer deployments end to end the founder way: make the repo agent-ready, wire the runtime layer, debug the whole seam in the field, and turn field learning into product changes I ship myself. Strong TypeScript and real git depth — branches, worktrees, rebases, and what breaks when many agents touch one repo.

Close to the OS: processes, file systems, shells, and CLIs are where my agents live. Practical AI coding agents in real work — not demos — with eval harnesses and fail-closed gates keeping parallel work honest. Written-first communication, self-directed, the bias toward shipping something real every week. Honest on scope: no weight fine-tuning claimed — eval-driven selection and routing is the proven part; founder/operator (three years, 2023–present), honestly stated.

CORE COMPETENCIES

Parallel-agent orchestration: many agents in parallel · git worktrees & branches · rebase discipline · review flow · what breaks when many agents touch one repo · race/conflict detection · recovery

Forward-deployed delivery: embedded delivery posture · customer discovery · sit-with-the-operator scoping · ambiguous-scope framing · rapid prototyping → production · handover discipline · field-to-product feedback

Engineering: TypeScript · Python · SQL · Bash · processes/file systems/shells/CLIs · REST APIs · CI/CD (GitHub Actions) · JSON Schema validation · Postgres

Agent quality: eval harnesses · golden datasets · behavioral assertions · LLM-as-a-judge with written rubrics · fail-closed recovery · trace intelligence · guardrails

EXPERIENCE

Founder / Applied AI Systems Architect 2023 – Present
GlacierEQ — Honolulu, HI

- Run parallel-agent orchestration as the native development pattern: many agents working real branches and worktrees against a live codebase — the same mechanical discipline Fredrin productizes for customers (own toolchain, not Fredrin itself).
- Own the end-to-end deployment motion the founder way: make repos agent-ready, author the project context and conventions that stop workers guessing, wire the runtime layer (agents, connectors, automations against APIs), and stay until the work is merging.
- Debug the whole seam: shells, worktrees, branches, CI status — instrument what breaks when agents touch shared repos and build the catchers: race/conflict detection, replay loops, fail-closed gates.
- Turn field learning into product changes shipped directly: reusable AI-engineering playbooks, eval harnesses, and integration patterns — every new engagement inherits the stack, not a blank slate.
- Build the integration layer behind the agents: **65** authored MCP connectors with typed contracts and structured validation — the connector discipline that turns recurring needs into reusable capabilities.

• Gate parallel work with evaluation: **989-test** suite, **62/62** verification checks green, behavioral assertions and written rubrics keeping agent output auditable — across an **1398-repository** estate (live-verified 2026-10-20).

EDUCATION

University of Hawaii at Manoa **2016**
Bachelor of Science

AWS Cloud Institute 2025
Cloud Computing program

Colorado State University
Undergraduate coursework

SELECTED PROOF

Parallel-agent orchestration as native pattern

• Multi-agent loops with typed handoff contracts, structured outputs, and fail-closed recovery — agents working real branches and worktrees against live codebases, three years.

Governed integration layer — 65 authored MCP connectors

• Governed tool/API/data-source connections with typed contracts — the runtime-layer wiring discipline, reusable across every engagement.

Field-to-product loop

• Embedded operator scoping: sit with the workflow, instrument it, rebuild it — field friction captured and shipped as product changes, not left as one-off scripts.

SKILLS

Languages: TypeScript · Python · SQL · Bash · JavaScript / Node.js

Git depth: branches · worktrees · rebases · multi-agent repo conflict discipline · CI integration

Agent systems: parallel agent orchestration · tool/function calling · MCP · structured outputs · eval harnesses · LLM-as-a-judge · fail-closed recovery

Production: REST APIs · Postgres · CI/CD (GitHub Actions) · observability · processes & shells · JSON Schema validation

Evidence boundary: portfolio repositories are independent work. Founder/operator depth (three years, 2023–present), honestly stated — no enterprise customer logos, no enterprise-account track record invented. No weight fine-tuning claimed — model strategy is eval-driven selection and routing. No Go / GCP / GraphQL / K8s / Terraform / data-science-statistics depth claimed. CRITICAL: no Fredrin usage claimed — the JD asks applicants to already use Fredrin; see REVIEW_REQUIRED.txt for the pre-application open call. Parallel-agent orchestration is his own toolchain, not Fredrin-internal experience.